

Vision Transformer

传统图像模型主要是 CNN，也就是 Convolutional Neural Network，卷积神经网络。Transformer 在 NLP 中已经证明了一件事：如果把输入看成一串 token，那么 Self-Attention 可以直接建模任意两个 token 之间的关系。

ViT 原论文《An Image is Worth 16x16 Words》提出：把图像切成固定大小的 patch，每个 patch 当成一个“视觉词”，再用纯 Transformer Encoder 做图像分类。论文指出，在大规模数据预训练后，纯 Transformer 可以在图像识别任务上取得很强效果。

(<https://arxiv.org/pdf/2010.11929>)

假设输入图像是： $X \in \mathbb{R}^{H \times W \times C}$ ，其中 H 是图像高度， W 是图像宽度， C 是通道数。RGB 图像中 $C = 3$

ViT 不直接把每个像素当成 token，因为那样 token 太多。它把图像切成大小为： $P \times P$ 的小块，也就是 patch。如果图像大小是 $H \times W$ ，patch 大小是 $P \times P$ ，那么 patch 数量为： $N = \frac{H}{P} \times \frac{W}{P}$ ，也可以写成 $N = \frac{HW}{P^2}$

[例如常见输入 $224 \times 224 \times 3$

如果 patch size 是 $P = 16$ ，那么每一行有 $\frac{224}{16} = 14$ 个 patch，每一列也有 14 个 patch，所以总 patch 数量是 $N = 14 \times 14 = 196$

这就是 ViT 标题中 “16x16 words” 的含义：每个 16×16 的图像块被当成一个视觉 token。]

每个 patch 的原始形状是 $P \times P \times C$

把它拉平成一个向量 $x_p^i \in \mathbb{R}^{P^2 C}$

其中 $i = 1, 2, \dots, N$ 表示第几个 patch。

如果 $P = 16, C = 3$ ，那么每个 patch 向量长度为 $P^2 C = 16 \times 16 \times 3 = 768$

但是 Transformer 需要的是统一维度的 token embedding。假设 Transformer 的隐藏维度是 D ，那么需要一个线性映射矩阵 $E \in \mathbb{R}^{P^2 C \times D}$ ，于是第 i 个 patch 的 embedding 是 $z_i = x_p^i E$ ，如果把所有 patch 组成矩阵 $X_p \in \mathbb{R}^{N \times P^2 C}$ ，那么 patch embedding 可以写成 $X_p E \in \mathbb{R}^{N \times D}$ ，所以到这里，图像已经从 $X \in \mathbb{R}^{H \times W \times C}$ 变成了 $Z \in \mathbb{R}^{N \times D}$ ，ViT 的第一步就是把二维图像转成一维 token 序列。

Patch embedding 也可以理解成一个卷积层，卷积核大小 $P \times P$ ，步长 P ，输出通道数 D ，对于第 (u, v) 个 patch，对应的第 d 个输出维度是：

$$y_{u,v,d} = \sum_{a=0}^{P-1} \sum_{b=0}^{P-1} \sum_{c=1}^C W_{a,b,c,d} X_{uP+a, vP+b, c} + b_d$$

每个 patch embedding 其实就是对一个 $P \times P$ 图像块做一次线性变换。所以 ViT 的 patch embedding 本质上就是把每个局部图像块压成一个 D 维向量。

在图像分类任务中，模型最后需要输出一个类别。

NLP 里的 BERT 有一个 [CLS] token, 最后用这个 token 表示整句话。ViT 也借用了这个思想, 加入一个可学习的分类 token: $x_{\text{cls}} \in \mathbb{R}^{1 \times D}$

然后把它拼到 patch token 前面:

$$Z_0 = [x_{\text{cls}}; x_p^1 E; x_p^2 E; \dots; x_p^N E]$$

此时序列长度从 N 变成 $N + 1$

为什么 class token 能代表整张图?

因为在后面的 Self-Attention 中, class token 会和所有 patch token 交互。经过多层 Transformer 后, class token 会逐渐聚合整张图的信息。最后分类时, 只取最后一层的 class token: z_L^0 送入分类头。

Transformer 的 Self-Attention 本身并不知道 token 的顺序。也就是说, 如果你把 patch 顺序打乱, Self-Attention 本身无法知道哪个 patch 在左上角, 哪个 patch 在右下角。

从数学上看, 假设输入 token 矩阵是 $Z \in \mathbb{R}^{M \times D}$, 其中 $M = N + 1$, 如果用一个置换矩阵 Π 打乱 token 顺序, 那么 $Z' = \Pi Z$

Self-Attention 的 Query、Key、Value 分别是 $Q = ZW_Q; K = ZW_K; V = ZW_V$, 打乱后 $Q' = \Pi Q; K' = \Pi K; V' = \Pi V$, 注意力分数变成:

$$Q' K'^T = (\Pi Q) (\Pi K)^T = \Pi Q K^T \Pi^T$$

也就是说, 打乱输入顺序后, 注意力矩阵只是被同步打乱了。模型并不知道“空间位置”本身。所以 ViT 加入可学习的位置编码 $E_{\text{pos}} \in \mathbb{R}^{(N+1) \times D}$, 最终输入 Transformer 的序列是 $Z_0 = [x_{\text{cls}}; X_p E] + E_{\text{pos}}$, 告诉模型每个 patch 在图像中的位置。

假设某一层 Transformer 的输入是 $Z \in \mathbb{R}^{M \times D}$

其中 $M = N + 1$ 表示 token 数量, D 表示每个 token 的维度。对于每个 token, 我们都生成三个向量 $q_i = z_i W_Q, k_i = z_i W_K, v_i = z_i W_V$, 分别叫做 Query, Key, Value, 矩阵形式为 $Q = ZW_Q; K = ZW_K; V = ZW_V$, 其中 $W_Q \in \mathbb{R}^{D \times d_k}; W_K \in \mathbb{R}^{D \times d_k}; W_V \in \mathbb{R}^{D \times d_v}$, 于是 $Q \in \mathbb{R}^{M \times d_k}; K \in \mathbb{R}^{M \times d_k}; V \in \mathbb{R}^{M \times d_v}$

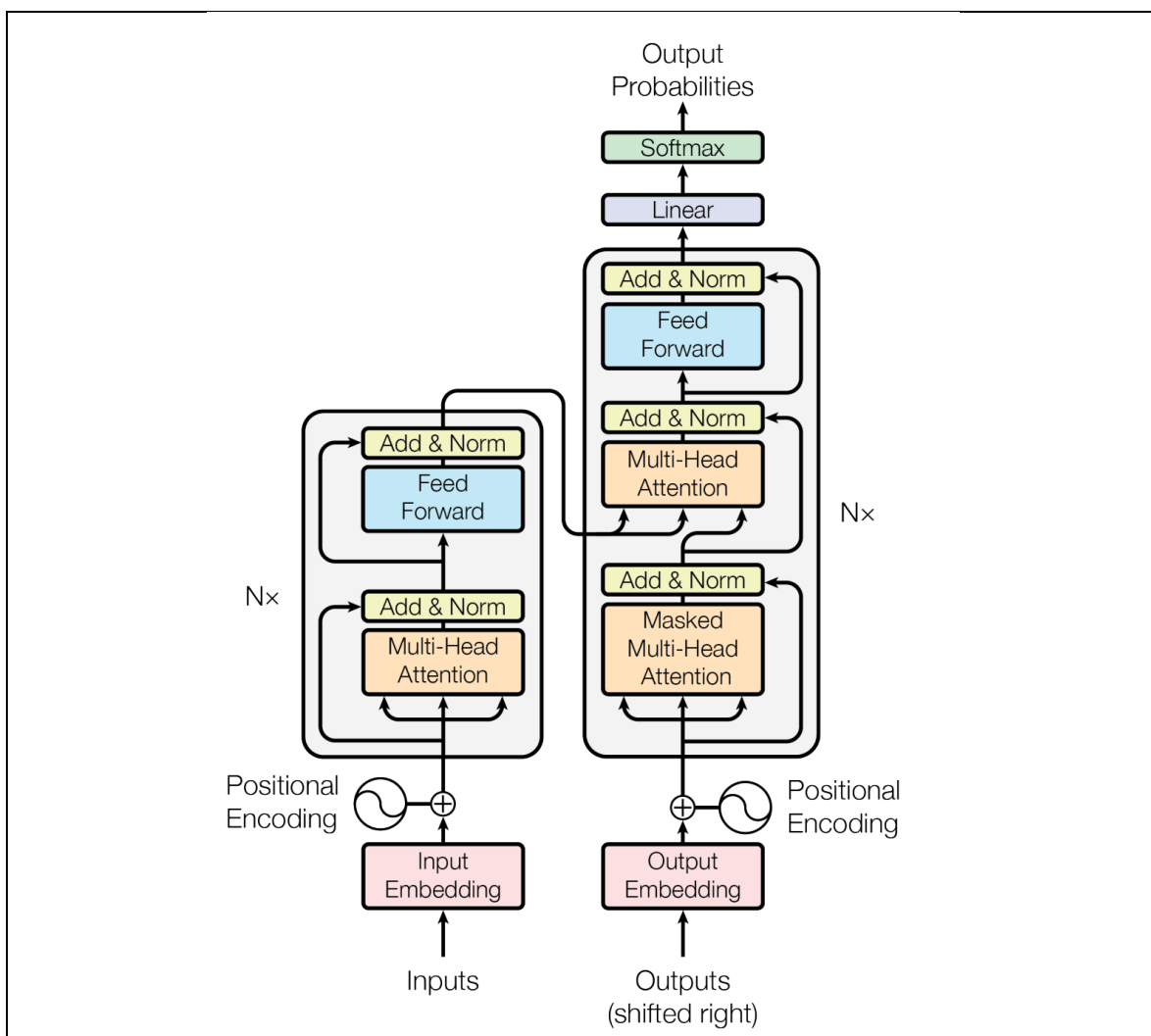
第 i 个 token 想知道自己应该关注第 j 个 token 多少, 就计算 $s_{ij} = q_i k_j^T$ 这就是两个向量的点积。

点积越大, 说明 q_i 和 k_j 方向越接近, 第 i 个 token 越应该关注第 j 个 token。矩阵形式就是 $S = QK^T$, 其中 $S \in \mathbb{R}^{M \times M}$, 第 i 行第 j 列 S_{ij} 表示第 i 个 token 对第 j 个 token 的关注程度。

Scaled Dot-Product Attention 的公式是 $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$

这个缩放因子来自 Transformer 原论文《Attention Is All You Need》。该论文提出了完全基于 Attention 的 Transformer, 并使用 scaled dot-product attention。

(<https://arxiv.org/pdf/1706.03762>)



为什么要除以 $\sqrt{d_k}$?

假设 $q_i = (q_{i1}, q_{i2}, \dots, q_{id_k})$, $k_j = (k_{j1}, k_{j2}, \dots, k_{jd_k})$

并且每一维近似满足 $\mathbb{E}[q_{ir}] = 0$, $\mathbb{E}[k_{jr}] = 0$, $\text{Var}(q_{ir}) = 1$, $\text{Var}(k_{jr}) = 1$

[方差的定义 $\text{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2$, 结合零均值假设, 我们可以推导出它们的二阶矩为 1, 即 $\mathbb{E}[q_{ir}^2] = 1$; $\mathbb{E}[k_{jr}^2] = 1$]

点积为 $q_i k_j^T = \sum_{r=1}^{d_k} q_{ir} k_{jr}$, 如果各维近似独立, 那么 $\text{Var}(q_i k_j^T) = \sum_{r=1}^{d_k} \text{Var}(q_{ir} k_{jr})$,

因为 $\text{Var}(q_{ir} k_{jr}) \approx 1$, 所以 $\text{Var}(q_i k_j^T) \approx d_k$ 也就是说, 当维度 d_k 很大时, 点积的数值会变得很大。

如果直接送入 softmax 会导致 softmax 非常尖锐。也就是某个位置接近 1, 其他位置接近 0。这样梯度容易变小, 训练不稳定。所以除以 $\sqrt{d_k}$

之后 $\text{Var}\left(\frac{q_i k_j^T}{\sqrt{d_k}}\right) = \frac{1}{d_k} \text{Var}(q_i k_j^T) \approx 1$, 这样注意力分数的尺度更稳定。

$[\text{Var}(q_i k_j^T) = \sum_{r=1}^{d_k} \text{Var}(q_{ir} k_{jr})]$ 中的 $q_i k_j^T$ 本质上是两个向量的点积，即：

$q_i k_j^T = \sum_{r=1}^{d_k} q_{ir} k_{jr}$ ，在概率论中，对于任意两个随机变量 X 和 Y ，它们和的方差为 $\text{Var}(X+Y) = \text{Var}(X) + \text{Var}(Y) + 2\text{Cov}(X,Y)$ ，因为我们前面假设了 q 和 k 的各个维度分量是**相互独立**的。这意味着对于不同的维度 $r \neq s$ 乘积项 $q_{ir} k_{jr}$ 与 $q_{is} k_{js}$ 之间也是相互独立的。对于独立的随机变量，它们的协方差 $\text{Cov} = 0$ 。

因此，多个独立随机变量之和的方差，等于它们各自方差的和。这就严格给出了 $\text{Var}\left(\sum_{r=1}^{d_k} q_{ir} k_{jr}\right) = \sum_{r=1}^{d_k} \text{Var}(q_{ir} k_{jr})$ ，现在，我们需要计算累加号里面每一项 $\text{Var}(q_{ir} k_{jr})$ 的具体数值。

根据两个独立随机变量乘积的方差公式 $\text{Var}(XY) = \mathbb{E}[(XY)^2] - (\mathbb{E}[XY])^2$

将 $X = q_{ir}$ 和 $Y = k_{jr}$ 代入。因为 q_{ir} 和 k_{jr} 相互独立，所以它们的期望的乘积等于乘积的期望： $\mathbb{E}[q_{ir} k_{jr}] = \mathbb{E}[q_{ir}] \cdot \mathbb{E}[k_{jr}] = 0 \times 0 = 0$

接着看平方项的期望 $\mathbb{E}[(q_{ir} k_{jr})^2]$ 。

同样因为独立性 $\mathbb{E}[(q_{ir} k_{jr})^2] = \mathbb{E}[q_{ir}^2] \cdot \mathbb{E}[k_{jr}^2]$

上述已推导出的二阶矩 $\mathbb{E}[q_{ir}^2] \cdot \mathbb{E}[k_{jr}^2] = 1 \times 1 = 1$

所以，单项乘积的方差为 $\text{Var}(q_{ir} k_{jr}) = \mathbb{E}[(q_{ir} k_{jr})^2] - (\mathbb{E}[q_{ir} k_{jr}])^2 = 1 - 0 = 1$

将单项方差代回求和公式得到 $\text{Var}(q_i k_j^T) = \sum_{r=1}^{d_k} \text{Var}(q_{ir} k_{jr}) = \sum_{r=1}^{d_k} 1 = d_k$

对第 i 个 token 来说，它对所有 token 的分数是 $s_{i1}, s_{i2}, \dots, s_{iM}$ ，softmax 得到权重 $a_{ij} = \frac{\exp(s_{ij})}{\sum_{t=1}^M \exp(s_{it})}$ ，其中 $a_{ij} \geq 0$ ，并且 $\sum_{j=1}^M a_{ij} = 1$ ，所以 a_{ij} 可以理解为第 i 个 token

从第 j 个 token 取信息的比例。

第 i 个 token 的输出是所有 Value 的加权和 $o_i = \sum_{j=1}^M a_{ij} v_j$ ，矩阵形式 $O = AV$ ，其中 $A = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)$ ，最终 $O = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$ ，这就是 Self-Attention。

对于 ViT 来说，每个 token 是一个 patch。假设第 i 个 patch 是图像左上角的一块，第 j 个 patch 是图像右下角的一块。CNN 需要很多层卷积才能让这两个远距离区域产生联系，因为卷积是局部操作。

但是 ViT 中，第 i 个 patch 可以在第一层 Self-Attention 中直接关注第 j 个 patch。所以 ViT 的优势是任意两个图像区域直接交互，即全局建模能力。原 ViT 论文也指出，Self-Attention 允许模型即使在较低层也能整合跨越整张图像的信息。

单个 Self-Attention 可以建模一种关系。但图像中有很多不同关系一部分 head 可以关注颜色相似区域；一部分 head 可以关注形状边界；一部分 head 可以关注物体整体结构；一部分 head 可以关注背景与前景关系。所以 ViT 使用 Multi-Head Self-Attention。

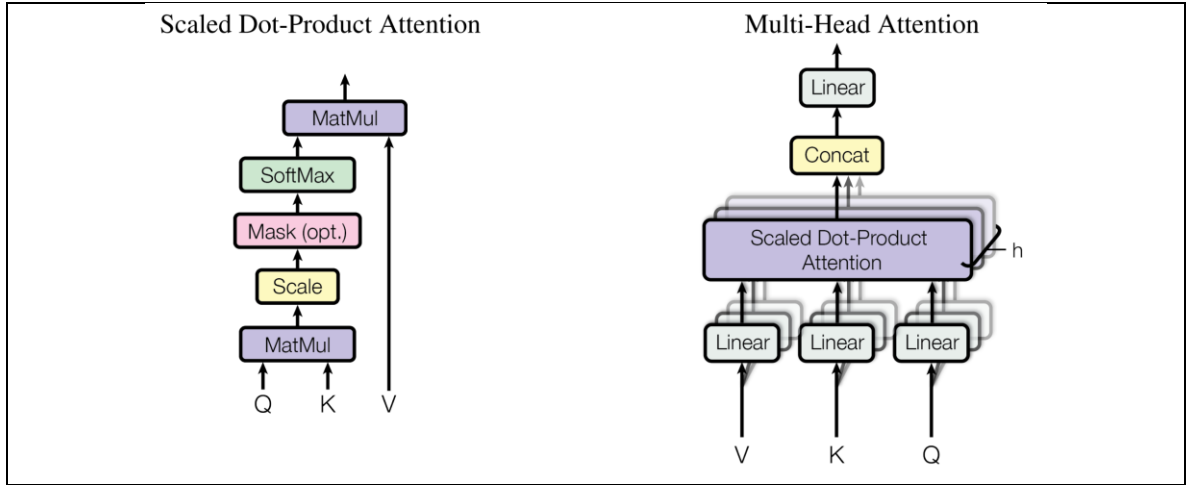
假设一共有 h 个 head。第 r 个 head 有自己的参数 $W_Q^{(r)}, W_K^{(r)}, W_V^{(r)}$

于是 $Q^{(r)} = ZW_Q^{(r)}; K^{(r)} = ZW_K^{(r)}; V^{(r)} = ZW_V^{(r)}$ ，第 r 个 head 的输出是：

$$\text{head}_r = \text{Attention}(Q^{(r)}, K^{(r)}, V^{(r)})$$

所有 head 拼接 $\text{Concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_h)$ ，再乘一个输出矩阵 W_O ，得到 $\text{MSA}(Z) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W_O$ ，如果隐藏维度是 D ，head 数是 h ，通常

每个 head 的维度是 $d_k = \frac{D}{h}$



[例如 ViT-B/16 常见配置中 $D = 768$ ， $h = 12$ ，所以每个 head 的维度是 $d_k = 64$ ，ViT-B/16 是 ViT 原论文中的经典模型配置之一。]

ViT 使用的是 Transformer Encoder，而不是 Decoder。一个 Encoder Block 主要包含两部分，第一个部分是 Muti-Head Self-Attention，第二部分是 MLP，并且每部分都有 LayerNorm+Residual Connection，ViT 中常见写法是 Pre-LN，也就是先 LayerNorm，再进入子层。

第 l 层输入是 Z_{l-1} ，先经过 Multi-Head Self-Attention:

$$Z_l' = Z_{l-1} + \text{MSA}(\text{LN}(Z_{l-1}))$$

然后经过 MLP:

$$Z_l = Z_l' + \text{MLP}(\text{LN}(Z_l'))$$

这就是一个完整的 Transformer Encoder Block。

对于一个 token 向量 $x \in \mathbb{R}^D$ ，LayerNorm 先计算均值 $\mu = \frac{1}{D} \sum_{i=1}^D x_i$ ，再计算方差

$\sigma^2 = \frac{1}{D} \sum_{i=1}^D (x_i - \mu)^2$ ，然后归一化 $\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$ ，最后加上可学习缩放和平移：

$$\text{LN}(x_i) = \gamma_i \hat{x}_i + \beta_i$$

其中 γ_i 和 β_i 是可学习参数。LayerNorm 的作用是让每个 token 的特征分布更稳

定，从而让训练更容易。

ViT 中的 MLP 一般是两层全连接网络 $\text{MLP}(x) = W_2 \sigma(W_1 x + b_1) + b_2$

其中 σ 通常是 GELU 激活函数。

如果输入维度是 D ，那么 MLP 的隐藏层维度通常是 $4D$ ，所以 $W_1 \in \mathbb{R}^{D \times 4D}$ ， $W_2 \in \mathbb{R}^{4D \times D}$ ，它的作用是对每个 token 单独做非线性特征变换。注意 Self-Attention 负责 token 之间的信息交互；MLP 负责 token 内部的特征变换。

现在把所有步骤串起来。输入图像 $X \in \mathbb{R}^{H \times W \times C}$

第一步，切 patch: $X \rightarrow X_p \in \mathbb{R}^{N \times P^2 C}$

第二步，patch embedding: $X_p E \in \mathbb{R}^{N \times D}$

第三步，加 class token: $[x_{\text{cls}}; X_p E] \in \mathbb{R}^{(N+1) \times D}$

第四步，加位置编码: $Z_0 = [x_{\text{cls}}; X_p E] + E_{\text{pos}}$

第五步，经过 L 层 Transformer Encoder:

$$Z'_l = Z_{l-1} + \text{MSA}(\text{LN}(Z_{l-1}))$$

$$Z_l = Z'_l + \text{MLP}(\text{LN}(Z'_l))$$

其中 $l = 1, 2, \dots, L$

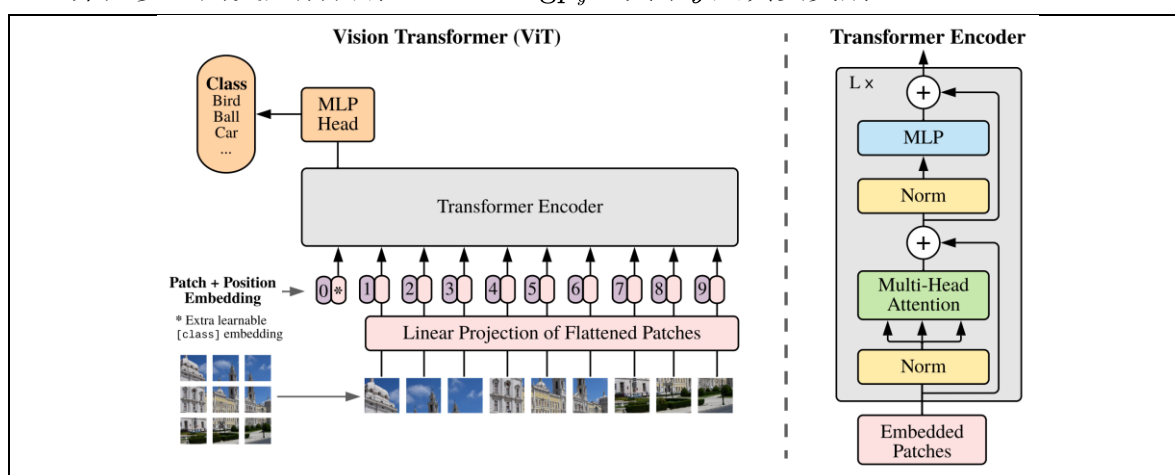
第六步，取最后一层 class token: $h_{\text{cls}} = Z_L^0$

第七步，分类头: $\text{logits} = h_{\text{cls}} W_{\text{cls}} + b_{\text{cls}}$

如果有 K 个类别，那么 $W_{\text{cls}} \in \mathbb{R}^{D \times K}$; $\text{logits} \in \mathbb{R}^K$

第八步，softmax 得到类别概率: $p_k = \frac{\exp(\text{logits}_k)}{\sum_{j=1}^K \exp(\text{logits}_j)}$

第九步，用交叉熵训练: $\mathcal{L} = -\log p_y$ ，其中 y 是真实类别。



[交叉熵梯度为什么是 $(p - y)$?

设分类 logits 为: $s_1, s_2, \dots, s_K$, softmax 输出 $p_k = \frac{e^{s_k}}{\sum_{j=1}^K e^{s_j}}$, 真实类别是 y , 交

叉熵损失 $\mathcal{L} = -\log p_y$, 也可以写成 one-hot 形式:

$$\mathcal{L} = - \sum_{k=1}^K y_k \log p_k$$

$$\text{其中 } y_k = \begin{cases} 1, & k = y \\ 0, & k \neq y \end{cases}$$

softmax + cross entropy 的经典结论是 $\frac{\partial \mathcal{L}}{\partial s_k} = p_k - y_k$, 这说明如果模型对正确类别的概率太低, 那么 $p_y - 1 < 0$, 梯度会推动正确类别 logit 上升。如果模型对错误类别的概率太高, 那么 $p_k - 0 > 0$ 梯度会推动错误类别 logit 下降。所以 ViT 最后的训练目标本质上就是让 class token 的输出更接近正确类别。]

CNN 的卷积公式可以写成 $y_{i,j,d} = \sum_{a,b,c} K_{a,b,c,d} x_{i+a,j+b,c}$, 这个公式说明 CNN 的一个输出位置只看附近局部区域。所以 CNN 的特点是局部连接; 参数共享; 平移等变性; 强图像先验。

ViT 的 Attention 公式是: $O = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$, 这个公式说明每个 patch 都可以和所有 patch 交互。所以 ViT 的特点是全局建模; 弱图像先验; 更依赖数据规模; 更容易迁移到多模态任务。

ViT 原论文结论: 需要 CNN 结构, 只要把图像切成 patch, 纯 Transformer 在大规模预训练后也可以表现很好。

CNN 自带很多适合图像的归纳偏置。

例如:

局部性: 相邻像素关系更重要;

平移等变性: 物体移动位置后, 特征仍然类似;

层级结构: 低层边缘, 中层纹理, 高层语义。

ViT 没有这么强的先验。它一开始并不知道相邻 patch 更相关, 也不知道图像有二维结构。虽然位置编码能提供位置信息, 但这些关系更多需要从数据中学出来。所以 ViT 通常需要更大规模数据预训练, 才能充分发挥效果。原 ViT 论文也强调了大规模预训练对 ViT 性能的重要性。